SIMT-Aware Lockstep Verification and Functional-Coverage Closure Methodology for an Open-Source RISC-V GPGPU: A Complete UVM 1.2 Environment

Samuel Moussa[a], Steven Ibrahim[a], Ahmad Sudky[a], Ahmad Fawzy[a], Abanoub Nabil[a], Alhassan Sayed[a], Hossam Hassan[b], Hyung-Min Yoon[c]

[a] *Department of Electronics and Communications Engineering, Faculty of Engineering, Minia, Egypt*
[b] *Independent Researcher, South Korea*
[c] *Siliconarts, Inc., South Korea*

Abstract

Open-source RISC-V GPGPUs ship without reference-model verification, functional-coverage models, or sign-off discipline, while concurrent fuzzing work demonstrates that real defects populate exactly this design class. This paper presents a complete UVM 1.2 verification environment and closure methodology for the Vortex RISC-V GPGPU, and the findings it produced on both sides of the comparison. The environment wraps a bus-master SIMT device with role-inverted agents, integrates the project's functional simulator as a per-configuration golden model over DPI-C, and renders verdicts through two checkers that are themselves qualified by permanent fault-injection guards: a bidirectional end-state scoreboard and a per-instruction, per-lane lockstep comparator governed by five alignment rules that we semi-

*Corresponding author.
Email address: samuel.moussa@mu.edu.eg (Hyung-Min Yoon)

formalize as precondition–soundness properties. A two-pass load-value feed makes fenceless multi-core programs instruction-granularity verifiable—verified modulo the fed race resolutions, with the assumption stated and, in the flagship case, discharged by a trace-level race witness. A three-layer coverage model adds what is, to our knowledge, the first published SIMT functional-coverage layer for RTL GPU verification (IPDOM divergence depth, scratchpad bank conflicts, coalescing classes), closed under machine-generated, RTL-cited exclusions enforced by a blocking waiver-integrity gate. The methodology surfaced externally corroborated RTL defects (a JALR LSB ISA deviation, a reset-relay X window behind a silenced assertion) and reference-model defects found by the lockstep itself. We report verification costs, per-generator marginal coverage, and the completed closure campaign [TBD: W1/W2/W3 numbers from the machine-work package].

Keywords: UVM, RISC-V, GPGPU, SIMT, functional verification, lockstep co-simulation, functional coverage, open-source hardware

1. Introduction

Open-source silicon has an industrial verification problem with a sharpening edge. The RISC-V ecosystem maintains mature verification assets for scalar cores—step-and-compare environments, constrained-random generators, ISA coverage libraries [5, 7, 6, 9, 4, 10]—but its most complex open processor class, the GPGPU, has had none of this. Vortex [1, 2], the leading open RISC-V GPGPU, ships a directed-kernel regression with no RTL-level reference checking and no functional-coverage model. Concurrently, FuzzGPU [3] demonstrated that a fuzzer can find real defects in exactly this

class of design (20 previously unknown bugs, 10 CVEs, across Vortex and Ventus), converting the gap from an academic inconvenience into a security exposure.

This paper closes the other half of that problem: not bug discovery, but *coverage-quantified sign-off*. Its contributions:

1. a complete, configuration-parametric UVM 1.2 [11] environment for a bus-master SIMT GPGPU (Section 3);
2. per-instruction, per-lane lockstep for SIMT via five alignment rules, *semi-formalized* as precondition–soundness properties (Section 5);
3. a two-pass load-value feed that makes fenceless multi-core programs instruction-granularity verifiable, with its soundness assumption stated and, where possible, discharged by trace-level race witnesses (Section 6);
4. a three-layer coverage model including the first published SIMT functional-coverage layer for RTL GPU verification (Section 8);
5. an exclusion methodology—machine-generated, RTL-cited waivers under a blocking hits-invariant merge gate—elevated here to a named, reusable contribution (Section 9);
6. an evidence-classified findings register covering both the RTL and the reference model, with dispositions and external corroboration (Sections 10–10.2), and the verification-cost and per-generator marginal-coverage evidence that adoption requires (Section 11).

All honesty constraints of the conference version carry over verbatim (Section 12); every number in this paper traces to a banked artifact or is marked as pending.

2. Background and related work

2.1. The DUT and its reference models

Vortex organizes as clusters → sockets → cores → warps → threads with per-core caches, optional shared L2/L3, an immediate-post-dominator (IPDOM) reconvergence stack, and hardware barriers; the verified build is RV32IMAF at a pinned revision on QuestaSim. Vortex has no trap architecture—a property with verification consequences (Section 10). SimX, its functional simulator, is structurally an RVVI-shaped golden model [6]; Spike [8] cannot execute the SIMT instructions and serves only as a scalar, independent base-ISA cross-check (11,076-retirement three-way audit: zero mismatches, injection-proven non-vacuity). *The SIMT axis has no independent reference*—a structural ceiling we return to in the threats section.

2.2. Reference-model verification of processors

Step-and-compare is the standard of care for RISC-V CPUs. core-v-verif [5] established the open-source pattern with ImperasDV [9] over RVVI [6]; riscv-dv [7] supplies constrained-random programs with a trace-compare flow; DiffTest [4] industrialized per-instruction architectural comparison over DPI-C for XiangShan; OpenTitan [10] demonstrates sign-off-grade UVM on open silicon at SoC scale. All target scalar or SIMD-vector cores. This work extends the family along three axes: a bus-master DUT with role-inverted agents, SIMT retirements (per-lane masked compare, split records, divergence), and weakly-ordered multi-core programs (the two-pass feed). Per-warp differential testing on this very DUT was concurrently demonstrated by FuzzGPU; our lockstep differs in the formalized alignment rules, the UVM

packaging, and its role as one qualified checker inside a sign-off flow rather than a fuzzer's oracle.

2.3. GPU kernel verification and testing

Software-level kernel verification is mature: GPUVerify [15] proves race- and divergence-freedom of OpenCL/CUDA kernels; GKLEE [16] performs concolic kernel testing, explicitly targeting divergent warps, non-coalesced accesses, and shared-memory bank conflicts—the same three phenomena our coverage layer quantifies at the RTL level; WEFT [17] verifies producer–consumer synchronization in GPU programs. Learned program generation has precedent in compiler fuzzing [18]. These operate on kernel source, not RTL; our contribution begins where their remit ends.

2.4. RTL GPU testing and memory consistency

FuzzGPU [3] (USENIX Security 2026) is the first RTL GPU fuzzer: SIMT-aware generation, trace-driven differential testing against ISA simulators on Verilator models of Vortex and Ventus, 20 bugs and 10 CVEs. Its evaluation is structural-coverage-only, its campaigns time-boxed, and it carries no functional-coverage model, no checker qualification, and no sign-off discipline—by design. Our methodology is the complementary half, and the flows cross-validate: the JALR LSB deviation we report (R1) was independently found by their S2 (filed against the project's instruction-set simulator; their RTL findings X1–X3, at a different pin, are disjoint from ours). Formal memory-consistency verification at RTL is addressed for CPU models by RTLCheck [21] and for GPU memory models in ASPLOS [22]; NVBitFI [20]

brought systematic fault injection to GPUs for reliability studies. Concurrently, the Ventus project developed GVM, a UVM-like framework for its own design.

2.5. Checker qualification

Mutation-based qualification—proving checkers can fail—is established practice [19]; we apply the discipline end-to-end to a SIMT lockstep flow, which is what allows our pass rates and coverage to be read at face value.

3. The verification environment

[Ported text: role-inverted agents; DPI-C SimX; probes; end-state and lockstep checkers; assertion-aware verdicts; configurability; protocol assertions. Expanded treatment of the probe layer's sampling semantics and of the launch-space sequences.]

4. Verdicts and non-vacuity

[Ported text.]

5. Per-instruction lockstep for SIMT: five rules, semi-formalized

5.1. The alignment problem

Let D be the DUT's record stream per (core, warp): records $d = (uuid, mask, PC, rd, val[\cdot])$, and G the reference's instruction stream $g = (ord, mask, PC, rd, val[\cdot])$ in program order. Comparing D and G naively produces false mismatches from five structural divergences. Each rule below is stated as *precondition* $\Rightarrow$

soundness property: if the precondition holds for the DUT, the transformation cannot mask a genuine defect (it may only remove false positives), and the residual comparator is exact outside the declared exception classes.

5.2. Rule 1 — retire order (sort by issue counter)

Precondition: the per-(core, warp) issue counter is strictly monotone in program order and does not wrap within a run. *Property:* sorting D by `uuid` per (core, warp) yields program order; hence record d_i aligns with instruction g_i by position. A retirement whose (PC, rd) differ from g_i remains visible. (Observed DUT behavior satisfies the precondition: an earlier-issued instruction was observed retiring after two later ones; the arbiter is `VX_commit.sv:56-71`.)

5.3. Rule 2 — cross-key (program-order keying)

Precondition: each stream independently enumerates the same per-(core, warp) instruction sequence. *Property:* aligning by per-(core, warp) ordinal position is well-defined without any shared identifier; the DUT `uuid`'s high bits supply (core, warp) attribution with no RTL change. Model-side identifiers (SimX leaves `uuid` at zero) are never used as keys.

5.4. Rule 3 — split retirements (mask-union aggregation)

Precondition: every record of one architectural instruction shares its `uuid`, and the union of the records' thread masks equals the instruction's active-lane set L (records may overlap; observed `0xd` then `0xe`). *Property:* aggregating records by `uuid` with mask union, then comparing per active lane, is equivalent to comparing the instruction itself: every lane in L is compared exactly once semantically, and no lane outside L contributes.

5.5. Rule 4 — load data (own tap, region-filtered)

Precondition: (i) a tap where true post-extension load values are observable (the commit arbiter's data field is stale for loads); (ii) a decidable predicate $V(a, v)$ that is true only when the two models' memory states agree at address a (initialized region, no preceding divergent write, reference value not initialization poison). *Property:* comparing only lanes with V is a sound *under*-approximation: it never masks a defect (non-V loads are deferred to the byte-exact end-state compare) at the cost of comparing fewer sites. (Empirically: 429 false mismatches on a known-good kernel without the filter; 74 compared / 113 filtered / 0 false with it.)

5.6. Rule 5 — volatile CSRs

Precondition: a statically known CSR class whose values are timing-defined rather than architecturally defined (`mcycle`/`minstret`/`mhpmcounter*`). *Property:* excluding their *data* (still comparing PC and destination) removes only comparisons with no architectural ground truth; soundness is preserved, completeness is reduced by exactly that class.

5.7. Declared exception classes and the composite claim

Beyond Rules 4–5, the comparator declares one bounded, logged tolerance: the DUT's `fsqrt.s` deviates by ≤ 1 ULP from correctly-rounded IEEE 754 [14], tolerated only for `fsqrt` (same-sign, finite, ≤ 1 ULP via `fp32_within_ulp`), tallied in `n_fp_ulp_tol`, with a two-pass reconvergence feed re-checking all downstream operations bit-exactly. The composite soundness claim: *if the preconditions of Rules 1–3 hold and both models faithfully*

execute the same program, then any retirement whose (PC, destination, per-lane value) differs from the reference on an active lane, outside the declared exception classes, is reported as a mismatch. The claim is conditional—the preconditions are DUT obligations an adopter must re-derive, listed as such in Section 12.

5.8. Validation

Lane-exact validation at five configurations (six programs) spanning a $2 \times 2 \times 3 \times 2$ axis space, including nested-divergence towers exercising Rule 3 through divergence and reconvergence; tallies and their provenance as in the conference version.

6. Verifying racy programs: the two-pass load-value feed

[Ported text with the assumption box and boundary discussion.]

7. Stimulus

[Ported text; simtgen axis status sentence updated after W4.]

8. A three-layer coverage model

[Ported text.]

9. Exclusion methodology: machine-generated, gate-checked waivers

[New text.]

10. Findings

10.1. RTL defects and hazards

[Ported text.]

10.2. Reference-model defects

[Ported text.]

11. Results

11.1. Completed closure campaign (fold-in)

[TBD: W1: new bank name/date, per-category table, totals vs. the relay-fix 94.72% baseline, simtgen closures folded (split-depth 4/4, bank 3/3, coalesce 3/3, vote_shfl [TBD: W4 outcome])]

11.2. Verification cost

Table 1: Wall-clock cost of the checking stack (median of three runs). [TBD: W2 measurements]

Program	Plain	Lockstep	Lockstep+feed	Peak RSS
vecadd	[TBD:]	[TBD:]	[TBD:]	[TBD:]
divergence kernel	[TBD:]	[TBD:]	[TBD:]	[TBD:]
fpu_test	[TBD:]	[TBD:]	[TBD:]	[TBD:]
memory kernel	[TBD:]	[TBD:]	[TBD:]	[TBD:]

11.3. Marginal coverage per generator kind

11.4. Bug-discovery dynamics

[TBD: W7: cumulative-findings curve over project milestones]

Table 2: Bins closed per program kind on the frozen model and the fold-in bank [TBD: W3-A: directed column + W1 columns].

Generator	Programs	New bins closed
Directed kernels	[TBD:]	[TBD:]
riscv-dv (seed farm)	90	0 (all categories bit-identical; toggle +0.06%)
simtgen (divergence/memory)	6	3 targets (isolated merge; fold-in pending)
simtgen (barrier/vote_shfl)	[TBD: W4]	[TBD: W4]

12. Threats to validity

12.1. Internal validity

The two-pass feed's soundness rests on the assumption that a divergent in-region load is a genuine race resolution; we state it, discharge it with trace-level witnesses where possible, and classify rather than force-green elsewhere. Differential testing is structurally blind to stimulus faults (shared-binary blindness); one input class is closed by independent assertions, the class is reported open. Comparator tolerances are enumerated (load filter, volatile CSRs, fsqrt 1-ULP) and logged, never silent.

12.2. External validity

All results are one DUT at one pin with locally modified files (disclosed), one simulator, one build; the alignment rules' DUT preconditions are stated so adopters can re-derive them; transfer beyond is belief, not demonstration. Open-source UVM simulation is maturing (upstream UVM 2017 elaborates in Verilator with constrained randomization), but SystemVerilog functional

coverage remains unsupported there, so the coverage-driven half requires a commercial simulator today.

12.3. Conclusion validity

Coverage figures are per-configuration and never blended; frozen banks are never overwritten; the D-extension elaboration is scoped by inspection (no functional-bin dilution; code-coverage impact conservative); per-mnemonic claims avoid aliased encodings; bank provenance and design state accompany every table.

13. Toward silicon sign-off

[Ported text.]

14. Conclusion

[Ported and updated once Section 11 is filled.]

Acknowledgment

The authors thank the Vortex group at Georgia Tech and the maintainers of the open verification assets (core-v-verif, riscv-dv, RVVI, riscvISACOV, Spike) leveraged by the environment.

References

[1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, H. Kim, Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics, in: Proc. 54th IEEE/ACM Int. Symp. Microarchitecture (MICRO-54), 2021, pp. 754–766.

[2] F. Elsabbagh et al., Vortex: OpenCL compatible RISC-V GPGPU, in: Workshop on Computer Architecture Research with RISC-V (CARRV), 2019.

[3] Z. Gao, J. Wang, Q. Zhou, L. Shan, J. Hu, X. Jia, Z. Lv, Fuzzing open-source GPU hardware with SIMT program generation, in: Proc. 35th USENIX Security Symposium, 2026, pp. 2465–2484.

[4] Y. Xu et al., Towards developing high-performance RISC-V processors using agile methodology, in: Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO-55), 2022.

[5] OpenHW Group, core-v-verif, `https://github.com/openhwgroup/core-v-verif`.

[6] OpenHW Group / Imperas, RVVI: RISC-V Verification Interface, `https://github.com/riscv-verification/RVVI`.

[7] CHIPS Alliance, riscv-dv, `https://github.com/chipsalliance/riscv-dv`.

[8] RISC-V International, Spike, the RISC-V ISA simulator, `https://github.com/riscv-software-src/riscv-isa-sim`.

[9] Imperas Software, ImperasDV, `https://imperas.com/imperasdv`.

[10] lowRISC, OpenTitan, `https://github.com/lowRISC/opentitan`.

[11] IEEE Standard for Universal Verification Methodology Language Reference Manual, IEEE Std 1800.2-2020, 2020.

[12] IEEE Standard for SystemVerilog, IEEE Std 1800-2017, 2017.

[13] A. Waterman, K. Asanović (Eds.), The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, 2019.

[14] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, 2019.

[15] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, P. Thomson, The design and implementation of a verification technique for GPU kernels, ACM Trans. Program. Lang. Syst. 37 (3) (2015) 10:1–10:49.

[16] G. Li, P. Gopalakrishnan, I. Ghosh, S. P. Rajan, G. Gopalakrishnan, GKLEE: Concolic verification and test generation for GPUs, in: Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP), 2012, pp. 215–224.

[17] T. L. Sharma, M. Bauer, A. Aiken, Verification of producer-consumer synchronization in GPU programs, in: Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 2015, pp. 88–98.

[18] C. Cummins, P. Petoumenos, H. Leather, S. Fursin, Compiler fuzzing through deep learning, in: Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA), 2018, pp. 95–105.

[19] L. Di Guglielmo, F. Fummi, G. Pravadelli, Vacuity analysis for property qualification by mutation of checkers, in: Proc. Design, Automation & Test in Europe (DATE), 2010, pp. 262–267.

[20] T.-T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa, M. B. Glick, S. W. Keckler, NVBitFI: Dynamic fault injection methodology for GPU applications, in: Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN), 2021, pp. 64–76.

[21] Y. A. Manerkar, D. Lustig, M. Martonosi, M. Pellauer, RTLCheck: A property checking approach for custom memory consistency models, in: Proc. 50th IEEE/ACM Int. Symp. Microarchitecture (MICRO-50), 2017, pp. 713–725.

[22] D. Lustig, S. Sahasrabuddhe, M. Giroux, A formal analysis of the NVIDIA PTX memory model, in: Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019, pp. 253–266.

[23] Siemens EDA, QuestaSim, `https://eda.sw.siemens.com/en-US/ic/questa/simulation/`.

[24] Verilator open-source project, `https://www.verilator.org`.

[25] Verilator project, UVM base libraries with edits for Verilator, `https://github.com/verilator/uvm`.

[26] PlanV, Enabling UVM support in Verilator (blog series), `https://planv.tech/blog/`.